



dbt on Databricks

What dbt is, why teams adopt it, how dbt Core and dbt Cloud compare on cost and setup effort - and a live demo on a synthetic GEA industrial telemetry fleet.

GEA Industrial Telemetry Showcase • raw → bronze → silver → gold • 800,956 synthetic IoT events

Agenda

Where we are going in the next 30 minutes

- **1. What is dbt?**
 - The transformation layer of the modern data stack - SQL, versioned and tested
- **2. Why use it?**
 - The concrete engineering problems it removes from a data team
- **3. dbt on Databricks**
 - Why the combination works: Delta, Unity Catalog, elastic SQL compute
- **4. dbt Core (CLI in your IDE) vs dbt Cloud**
 - Capabilities, licence cost and realistic setup effort for both
- **5. Live demo on this repository**
 - Seeds, incremental builds, lineage-based selection, tests, docs - with real output

TAKEAWAY

Everything shown in the demo section was executed against this repo; output is copied verbatim.

01

What is dbt?

The 'T' in ELT - transformation as software engineering

dbt in one slide

data build tool - an open-source transformation framework

- **You write SELECT statements, dbt handles everything around them**
 - Each model is a .sql file containing one SELECT; dbt wraps it in the right DDL
 - Jinja templating adds variables, macros, conditionals and reuse to plain SQL
- **dbt compiles to native warehouse SQL and pushes execution down to Databricks**
 - No data leaves the platform - dbt orchestrates, the warehouse computes
- **Dependencies are declared, never hardcoded**
 - `ref('silver_fact_device_events')` resolves to the real catalog.schema.table at runtime
 - From those references dbt infers the full DAG and the correct execution order
- **Tests, documentation and lineage live in the same repository as the SQL**
 - Everything is a plain text file: reviewable in pull requests, versioned in git

TAKEAWAY

dbt turns analytics SQL into software: modular, tested, documented and reproducible.

Why teams adopt dbt

The problems it removes, and what replaces them

Without dbt

- **Hundreds of ad-hoc SQL scripts and notebooks**
 - Run order kept in someone's head or a fragile scheduler
- **Hardcoded table names across environments**
 - Dev/prod drift, manual find-and-replace before every release
- **Data quality checked after the complaint arrives**
 - No automated regression net when logic changes
- **Documentation in a wiki that went stale months ago**
 - Nobody knows what breaks if a column changes
- **Full table rebuilds because nobody trusts partial loads**

With dbt

- **One project, one dependency graph, one command**
 - dbt build runs models and tests in dependency order
- **Environment-agnostic references via ref() and source()**
 - Same code promotes from dev to CI to prod unchanged
- **Tests are declared next to the model and run every build**
 - not_null, unique, relationships, accepted_values, custom SQL
- **Docs and lineage generated from the code itself**
 - Impact analysis before you merge, not after you break it
- **Incremental materializations process only new data**

TAKEAWAY

The value is not 'nicer SQL' - it is engineering discipline applied to analytics.

The building blocks you will see today

Core dbt concepts used throughout this repository

- **Models**
 - SELECT statements materialized as view, table or incremental table
- **Seeds**
 - Small CSV files versioned in the repo and loaded with dbt seed - deterministic demo input
- **Tests**
 - Generic tests declared in YAML plus singular tests written as SQL that must return zero rows
- **Macros**
 - Reusable Jinja functions - this repo uses them for schema routing, batch logging and a cross-adapter day spine
- **Materializations and hooks**
 - Incremental strategies, post-hooks for watermark maintenance and batch execution logging
- **Packages, docs and exposures**
 - dbt_utils and dbt_expectations for extra tests; doc blocks for reusable descriptions

02

dbt on Databricks

Why this specific combination

Why dbt and Databricks fit together

Lakehouse execution plus declarative transformation

What Databricks brings

- **Delta Lake tables with ACID transactions and time travel**
 - MERGE support that makes incremental models and SCD2 practical
- **Unity Catalog governance**
 - Three-level catalog.schema.table namespace, lineage and access control
- **Elastic SQL warehouses**
 - Scale compute per workload; serverless starts in seconds
- **One platform for batch, streaming and ML on the same tables**

What dbt adds on top

- **Structure: medallion layers enforced by folder-level configuration**
 - raw → bronze → silver → gold, each with its own schema and defaults
- **Dependency management and safe partial rebuilds**
 - Rebuild an exact slice of the DAG instead of the whole warehouse
- **Automated data quality gates on every run**
- **A living catalog with column-level documentation**
- **Environment promotion through profiles and targets**

TAKEAWAY

dbt never moves data - it compiles SQL that Databricks executes on your own compute.

The demo project architecture

Synthetic GEA machine fleet: slicers, cookers, bakers and packers



WIRED WITH ref() ONLY

No model references a physical table name. Every hop uses ref(), so dbt can infer the complete lineage graph and rebuild any slice of it on demand.

The project by the numbers

Reported by dbt itself at parse time and after a full build

19

models across bronze, silver and gold

80

data tests declared in YAML and SQL

4

seeds - deterministic CSV inputs

800,956

raw synthetic device events

103

nodes passing in a full build

- **Bronze reduces 800,956 raw rows to 800,691 clean rows**
 - 265 rows removed: injected duplicate event_id values and rows with a null machine_id - the unique and not_null tests pass because of the transformation, not by luck
- **silver_dim_machines holds 451 SCD2 versions for 20 machines**
 - Exactly one current row per machine, with no overlapping validity periods (enforced by a singular test)

Figures captured from dbt parse and dbt build --full-refresh on the local target.

03

Core or Cloud?

Two ways to run the exact same project

Option A: dbt Core in your IDE

Open-source CLI, Apache 2.0 licence, runs anywhere Python runs

How it works

- **pip install dbt-core dbt-databricks in a virtual environment**
- **Connection details live in profiles.yml with credentials from environment variables**
- **You develop in VS Code with the dbt Power User extension, git and a terminal**
- **You choose the orchestrator**
 - Databricks Workflows has a native dbt task, or use Airflow / GitHub Actions
- **Docs are static files you host yourself**

What you own

- **Python environment and adapter version management**
- **Secret handling for warehouse tokens**
- **Scheduling, retries, alerting and run history**
- **CI pipeline: dbt build on pull requests, state comparison, artifact storage**
- **Hosting for dbt docs and the manifest**
- **Onboarding: every analyst needs a working local setup**

TAKEAWAY

Maximum control and zero licence cost - in exchange for owning the platform plumbing.

Option B: dbt Cloud

Managed development, scheduling and collaboration on top of the same dbt project

What you get

- **Browser-based IDE with git guardrails - no local setup for analysts**
- **Managed job scheduler with logs, retries, notifications and run history**
- **Hosted documentation and lineage, always current**
- **CI jobs that build only modified models on a pull request**
- **Governance features on higher tiers**
 - SSO, RBAC, audit logging, dbt Mesh across multiple projects
- **Semantic layer, catalog and Copilot code generation on paid tiers**

What to check before buying

- **Seat count:** pricing is per developer seat, per month
- **Model build volume:** plans cap successful models built per month
- **Project count:** Starter allows a single project
- **Network access from dbt Cloud to your Databricks workspace**
 - PrivateLink and IP restrictions are Enterprise+ features
- **Where secrets and artifacts are stored under your security policy**

TAKEAWAY

Same models, same tests, same git repo - the difference is who runs and operates it.

Licence cost comparison

dbt pricing published by dbt Labs, plus the Databricks compute you pay either way

Option	Licence cost	Included limits	Best fit
dbt Core	Free, Apache 2.0 open source	No limits imposed by dbt; you provide the runtime and orchestration	Small, technical teams comfortable owning CI/CD and scheduling
dbt Developer	Free	1 developer seat, 3,000 successful models built per month, 1 project	Individual evaluation or a single-person analytics function
dbt Starter	\$100 per user / month	5 developer seats, 15,000 models built per month, 1 project, API access	A first production project with a small team and no local setup
dbt Enterprise	Custom pricing, billed annually	100,000 models built per month, 30 projects, Mesh, advanced catalog and semantic layer	Multiple domains and teams needing governance and SSO
dbt Enterprise+	Custom pricing	Unlimited projects, PrivateLink, IP restrictions, hybrid projects	Regulated environments with strict network and security requirements
Databricks compute	Pay per DBU plus cloud infrastructure	Charged identically whether dbt Core or dbt Cloud submits the SQL	Usually the dominant cost line - optimise warehouse size and auto-stop

dbt plan details from getdbt.com/pricing (checked July 2026). Databricks DBU rates vary by region, cloud and warehouse type - confirm with your account team.

Setup effort comparison

What it actually takes to get a team productive

Dimension	dbt Core in an IDE	dbt Cloud
Initial environment	Python, virtual environment, adapter install, profiles.yml and a personal access token per developer	Log in, connect the Databricks workspace once, invite users
Skills assumed	Comfortable with git, terminal and Python packaging	SQL and basic git; the IDE provides branch and commit guardrails
Scheduling	Build it: Databricks Workflows dbt task, Airflow or GitHub Actions	Built-in scheduler with logs, retries and notifications
CI on pull requests	Write the workflow yourself, including state comparison and artifact handling	Configure a CI job; slim builds of modified models are native
Documentation hosting	Generate static files and host them somewhere	Hosted and refreshed automatically after each run
Ongoing maintenance	Version upgrades, dependency drift and runner maintenance are yours	Managed by dbt Labs; you control when to adopt new releases
Typical time to first model	Hours per developer, plus days to build the surrounding CI/CD platform	Minutes per developer once the workspace connection exists

Both options run the identical project - this repository can be executed either way without changing a single model.

Choosing between them

A pragmatic decision guide

- **Start with dbt Core when**

- You already run Databricks Workflows or Airflow and have CI/CD conventions in place
- The team is small, technical, and licence budget is constrained
- You want the demo running today with zero procurement

- **Move to dbt Cloud when**

- Analysts without local tooling need to contribute safely
- You need audited run history, alerting and SSO without building it
- Multiple teams publish interdependent projects and need Mesh-style governance

- **A common middle path**

- Develop with dbt Core locally, orchestrate production with the native dbt task in Databricks Workflows - no extra licence, scheduling handled by the platform you already pay for

TAKEAWAY

The project code is identical in all three paths; only the runtime and operations differ.

04

Live demo

Real commands, real output, from this repository

Demo flow

Follow along in `docs/dbt_presentation_demo.md`

- **1. Deterministic inputs**
 - dbt seed loads the four versioned CSV files that make up the raw layer
- **2. A full build of the whole DAG**
 - dbt build runs 19 models and 80 tests in dependency order
- **3. Incremental re-runs**
 - Re-running bronze with unchanged input does almost no work
- **4. Lineage-based selection**
 - dbt build --select +model+ rebuilds an exact upstream and downstream slice
- **5. Tests as a quality gate**
 - What a passing run looks like, and what a real failure looks like
- **6. Documentation and lineage**
 - dbt docs generate produces a browsable catalog and DAG
- **7. The business output**
 - Gold views queried directly on the warehouse

Demo 1 - Deterministic inputs

Seeds ship with the repository, so every reviewer builds the same dataset

```
$ dbt seed
```

ACTUAL OUTPUT FROM THIS REPO

```
12:42:04 Found 19 models, 80 data tests, 4 seeds, 1 exposure, 1157 macros
12:42:04 Concurrency: 8 threads (target='dev')

12:42:10 1 of 4 START seed file raw.raw_device_events ..... [RUN]
12:42:24 4 of 4 OK loaded seed file raw.raw_machines ..... [CREATE 20 in 13.56s]
12:42:24 2 of 4 OK loaded seed file raw.raw_error_codes ..... [CREATE 18 in 13.56s]
12:42:24 3 of 4 OK loaded seed file raw.raw_event_types ..... [CREATE 10 in 13.56s]
12:53:26 1 of 4 OK loaded seed file raw.raw_device_events ..... [CREATE 800956 in 675.61s]

12:53:26 Finished running 4 seeds in 0 hours 11 minutes and 22.38 seconds (682.38s).
12:53:26 Completed successfully
12:53:26 Done. PASS=4 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=4
```

- 800,956 events, 20 machines, 10 event types and 18 error codes - generated from a fixed random seed so the dataset is reproducible on any machine
- The raw data deliberately contains duplicate event ids and rows with a null machine id, so the bronze layer has something real to clean
- Seeds are for small, versioned reference data; 800k rows over a SQL warehouse takes 11 minutes, so production volumes belong in a proper ingestion path, not in dbt seed

Demo 2 - One command builds the whole DAG

dbt build runs models, then their tests, in dependency order

```
$ dbt build --full-refresh --threads 1
```

ACTUAL OUTPUT FROM THIS REPO

```
13:11:20 Found 19 models, 80 data tests, 4 seeds, 1 exposure, 1157 macros
13:11:20 Concurrency: 1 threads (target='dev')

13:12:51 36 of 103 OK created sql table model silver.silver_dim_machines ..... [OK in 3.78s]
13:12:56 37 of 103 OK created sql table model silver.silver_fact_device_events . [OK in 5.23s]
13:13:05 40 of 103 PASS assert_no_overlapping_scd2_periods ..... [PASS in 1.88s]
13:13:42 59 of 103 OK created sql table model silver.silver_fact_error_events .. [OK in 3.98s]
13:13:49 60 of 103 OK created sql view model gold.agg_event_frequency_day ..... [OK in 7.50s]

13:15:14 Finished running 1 exposure, 7 incremental models, 3 seeds, 3 table models,
      80 data tests, 9 view models in 0 hours 3 minutes and 53.67 seconds (233.67s).
13:15:14 Completed successfully
13:15:14 Done. PASS=102 WARN=0 ERROR=0 SKIP=0 NO-OP=1 TOTAL=103
```

- dbt worked out the order: seeds first, then bronze, silver, gold - and each model's tests immediately after the model itself
- A single exit code tells CI whether the warehouse is in a trustworthy state
- Run on a Databricks Free Edition serverless SQL warehouse against the gea_demo catalog; the 800k-row seed was loaded separately beforehand

Demo 3 - Incremental models make re-runs cheap

Same command, unchanged input: bronze does almost no work

```
$ dbt build --select tag:bronze
```

ACTUAL OUTPUT FROM THIS REPO

```
13:15:33 Found 19 models, 80 data tests, 4 seeds, 1 exposure, 1157 macros
13:15:33 Concurrency: 1 threads (target='dev')

13:15:52 1 of 20 OK created sql incremental model bronze.bronze_device_events . [OK in 10.33s]
13:16:01 2 of 20 OK created sql incremental model bronze.bronze_error_codes ... [OK in 9.38s]
13:16:11 3 of 20 OK created sql incremental model bronze.bronze_event_types ... [OK in 9.21s]
13:16:20 4 of 20 OK created sql incremental model bronze.bronze_machines ..... [OK in 9.23s]
13:16:43 18 of 20 PASS unique_bronze_machines_machine_id ..... [PASS in 1.48s]

13:16:46 Finished running 4 incremental models, 16 data tests in 0 hours 1 minutes
and 13.39 seconds (73.39s).
13:16:46 Completed successfully
13:16:46 Done. PASS=20 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=20
```

- **bronze_device_events** re-ran over an 800,691-row Delta table in 10.33s because its delta filter matched no new events - it inserted nothing rather than rebuilding the table
- The elapsed time here is dominated by per-statement round-trips to a Free Edition serverless warehouse, not by data processing
- The same filter is what keeps cost flat as history grows: incremental models never re-scan data they have already loaded

Demo 4 - Rebuild exactly the slice you touched

The + operator walks the lineage graph upstream and downstream

```
$ dbt build --select +silver_fact_device_events+
```

ACTUAL OUTPUT FROM THIS REPO

```
13:16:59 Found 19 models, 80 data tests, 4 seeds, 1 exposure, 1157 macros
13:16:59 Concurrency: 1 threads (target='dev')

13:19:49 65 of 66 PASS accepted_values_agg_error_analysis_severity ..... [PASS in 2.17s]
13:19:52 66 of 66 PASS not_null_agg_error_analysis_machine_type ..... [PASS in 1.55s]

13:19:52 Finished running 3 incremental models, 2 seeds, 2 table models,
52 data tests, 7 view models in 0 hours 2 minutes and 53.80 seconds (173.80s).
13:19:52 Completed successfully
13:19:52 Done. PASS=66 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=66
```

- 66 of the 103 nodes were selected: everything the model depends on, the model itself, and everything downstream of it - the rest were correctly left alone
- Nobody maintains that list by hand; it is derived from the ref() calls in the SQL
- Same syntax scales in CI: --select state:modified+ rebuilds only what a pull request actually changed

Demo 5 - A failure stops the build, it does not leak

A real defect this repository hit on Databricks, caught by dbt itself

```
$ dbt build --full-refresh # 8 threads
```

ACTUAL OUTPUT FROM THIS REPO

```
13:09:23 Completed with 2 errors, 0 partial successes, and 0 warnings:
```

```
13:09:23 Failure in model agg_event_frequency_hour (models\gold\agg_event_frequency_hour.sql)
```

```
13:09:23 Database Error in model agg_event_frequency_hour
[DELTA_CONCURRENT_APPEND.ROW_LEVEL_CHANGES] Transaction conflict detected.
A concurrent MERGE added data to table gea_demo.silver.silver_tec_watermark
committed at version 3. The concurrent operation modified the same rows that
this transaction attempted to modify. Please retry the operation.
```

```
13:09:23 Done. PASS=89 WARN=0 ERROR=2 SKIP=12 NO-OP=1 TOTAL=104
```

- Three gold models share one watermark table and advance it from a post-hook, so running them in parallel makes their MERGE statements collide on the same Delta rows
- 12 downstream nodes were skipped rather than built on top of a half-written table, and the non-zero exit code fails the pipeline
- Failing data tests behave exactly the same way: the run stops and names the offending model, so bad data never reaches a dashboard

Serialising the run (--threads 1) produced PASS=102 ERROR=0; the durable fix is to key the watermark by model.

Demo 6 - Documentation and lineage from the code

Descriptions live in reusable doc blocks and are compiled into a catalog

```
$ dbt docs generate && dbt docs serve
```

ACTUAL OUTPUT FROM THIS REPO

```
13:20:03 Running with dbt=1.11.12
13:20:04 Registered adapter: databricks=1.12.2
13:20:05 Found 19 models, 80 data tests, 4 seeds, 1 exposure, 1157 macros
13:20:05 Concurrency: 8 threads (target='dev')

13:20:11 Building catalog
13:20:14 Catalog written to C:\_REPO\PoC\dbt_databricks_industrial_iot\target\catalog.json
```

- **dbt docs serve** opens an interactive DAG: click any node to see its SQL, columns, tests and every downstream consumer
- Column descriptions are defined once in docs/column_decription.md and referenced with `{{ doc('column_machine_id') }}`, so the same definition appears on every layer
- The exposure declared for the raw layer gives the graph an explicit entry point

Demo 7 - What the pipeline actually produced

Querying the built warehouse directly after the run

```
$ select * from gold.dim_machine_status / gold.agg_error_analysis
```

ACTUAL OUTPUT FROM THIS REPO

```
== Row counts by layer (gea_demo on Databricks) ==
```

object	rows	
raw.raw_device_events	800956	
bronze.bronze_device_events	800691	<- 265 dirty rows removed by bronze
silver.silver_fact_device_events	800691	
silver.silver_dim_machines	451	<- SCD2 versions for 20 machines
silver.silver_fact_error_events	100	

```
== gold.dim_machine_status (sample) ==
```

machine_id	machine_type	status_value	plant_location	status_since
BKR-001	BAKER	RUNNING	Shanghai	2026-06-26 10:05:21+00:00
BKR-003	BAKER	MAINTENANCE	Bologna	2026-06-26 19:25:52+00:00
CKR-001	COOKER	RUNNING	Cologne	2026-06-29 06:30:52+00:00

```
== gold.agg_error_analysis (sample) ==
```

machine_type	severity	error_count	avg_resolution_s	maintenance_required
PACKER	LOW	12	1110.0	0
BAKER	HIGH	10	14250.0	10
COOKER	HIGH	10	21996.0	10

Gold views read only from silver; identical results were produced by the same code on DuckDB.

The commands worth remembering

Copy-paste cheat sheet for the audience

- **dbt build**
 - Run every model and every test in dependency order
- **dbt build --select +silver_fact_device_events+**
 - Rebuild one model plus all of its ancestors and descendants
- **dbt build --select tag:gold**
 - Rebuild a single layer using folder-level tags
- **dbt build --select state:modified+**
 - In CI, rebuild only what changed in the pull request and everything downstream
- **dbt test / dbt compile --select <model>**
 - Run quality gates alone, or inspect the exact SQL dbt sends to Databricks
- **dbt docs generate && dbt docs serve**
 - Publish the lineage graph and column-level catalog

TAKEAWAY

Full walkthrough with talking points: docs/dbt_presentation_demo.md

Takeaways

What to remember when the slides are gone

- **dbt makes analytics SQL behave like software**
 - Versioned, reviewed, tested, documented and reproducible by anyone who clones the repo
- **Databricks executes everything - dbt only orchestrates and compiles**
 - No data movement, no extra runtime, full use of Delta and Unity Catalog
- **ref() is the whole trick**
 - Declared dependencies give you lineage, correct run order and surgical partial rebuilds
- **Incremental materializations keep re-runs cheap as history grows**
 - bronze_device_events re-ran over 800,691 rows without inserting anything, because its delta filter matched no new events
- **dbt Core costs nothing but you own the platform; dbt Cloud sells you that platform**
 - Free Developer tier, \$100 per seat on Starter, custom Enterprise - the project code is identical either way

TAKEAWAY

Everything in this deck was run against Databricks Unity Catalog (catalog gea_demo).



Questions?

Repository: `dbt_databricks_industrial_iot` • Demo guide: `docs/dbt_presentation_demo.md`
• Architecture specification: `CLAUDE.md`

All command output in this deck was captured from real runs of this project.